Real-Time LAN – Controller Area Network (CAN)

Presented by
Souradeep Bhattacharya

Instructor: **Prof. Manimaran Govindarasu**
CprE 4580/5580: Real-Time Systems

Department of Electrical and Computer Engineering

IOWA STATE UNIVERSITY

Presentation Outline

- Introduction to LAN and Fieldbus
- Overview of CAN
- Working of CAN
- Project Demo
- Example Project Ideas

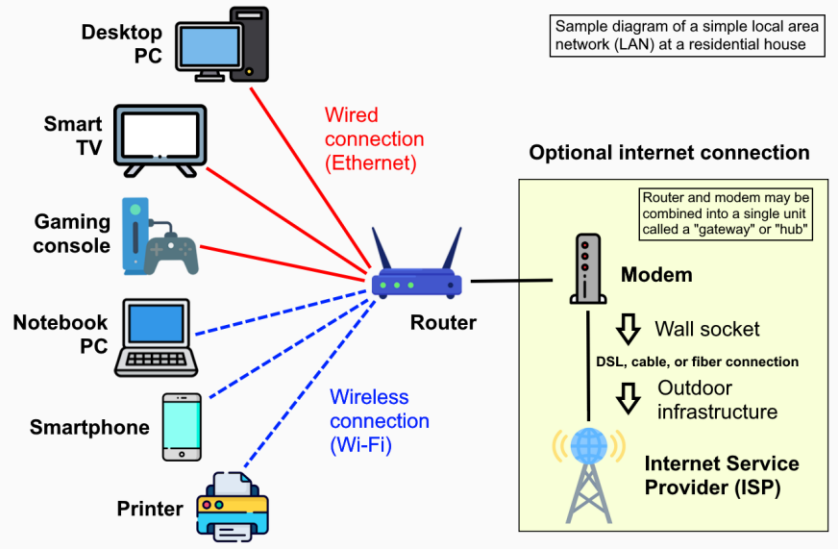
LAN – Basic Concepts

A local area network (LAN) interconnects network equipment within a limited area.

Ethernet and Wi-Fi are the two most common technologies used for LAN.

Key Features:

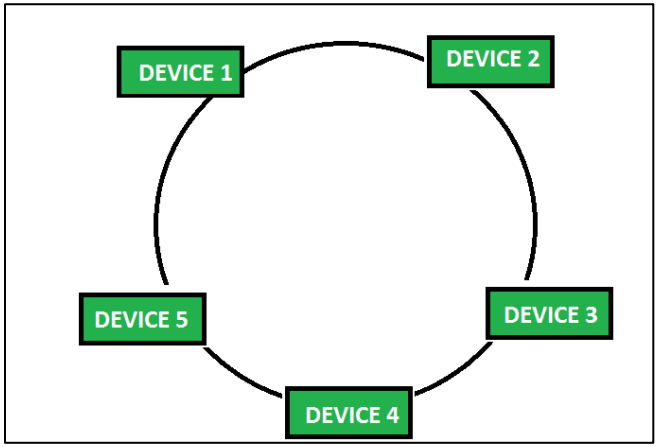
- *Limited Coverage*
- *High Data Transfer Speeds*
- *Resource Sharing*
- *Secure and Controlled Access*



Ref: https://en.wikipedia.org/wiki/Local_area_network

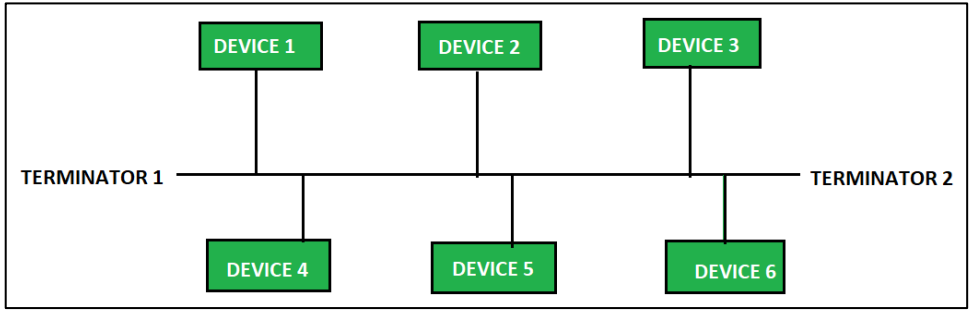
Common Topologies in LAN

Ring Topology



- Low Cost
- Efficient Data Transfer
- Single Point of Failure

Bus Topology



- Easy Installation
- Less Amount of Cable
- Number of supported devices limited by the length of the backbone cable

Application of LAN in ICS Environments

LAN communication technologies and products used in:

- Intra-vehicular communication
- Robotics
- Process control industries
- Smart Grid and Energy Systems
- Maritime and Railway Systems
- Building Automation

Bus topology (Fieldbus) is most common in **legacy and embedded industrial control networks**

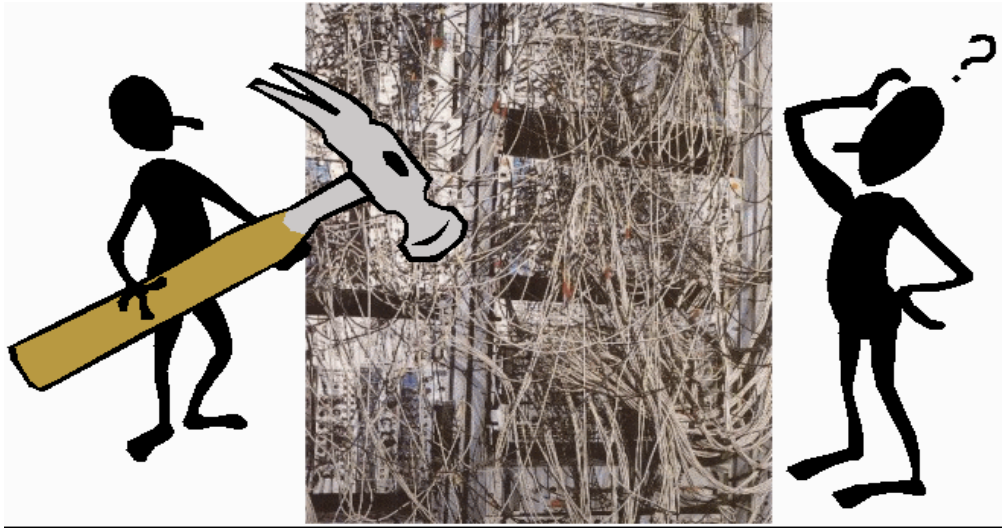
How do we connect these control devices?

- With conventional systems, data is exchanged by means of dedicated signal lines or wires.
- But this is becoming increasingly difficult and expensive as control functions become ever more complex.
- In the case of complex control systems in particular, the number of connections cannot be increased much further.
- Solution: Use Fieldbus networks for connecting the control devices

Fieldbus Networks: basic motivation

Why use Fieldbus Networks?

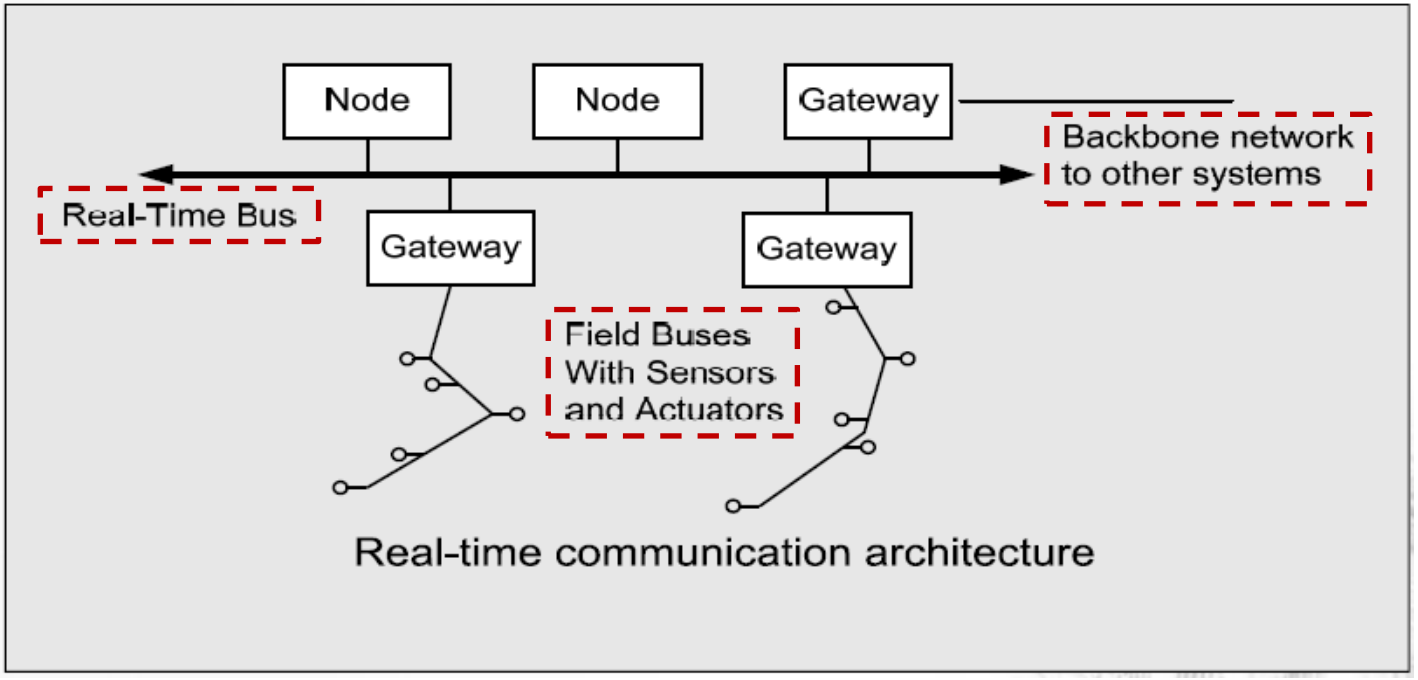
To avoid this…



Traditional Wiring - *two pairs of cables can substitute all typical connections*.

Fieldbus Real-Time Communication Architecture

Three different communication networks in real-time application.



Fieldbus Networks

Fieldbuses are communication technologies and products used in vehicular, automation, and process control industries.

1) Proprietary Fieldbuses (Closed Fieldbuses)

Proprietary Fieldbuses are the intellectual property of a particular company or body.

2) Open Fieldbuses

For a Fieldbus to be Open, it must satisfy the following criteria.

a) The full Fieldbus Specification must be published and available at a reasonable price.

b) Critical ASIC components must be available, also at a reasonable price.

c) Well defined validation process, open to all of the Fieldbus users.

Fieldbus Advantages

1) Reduces the complexity of the control system in terms of hardware outlay.

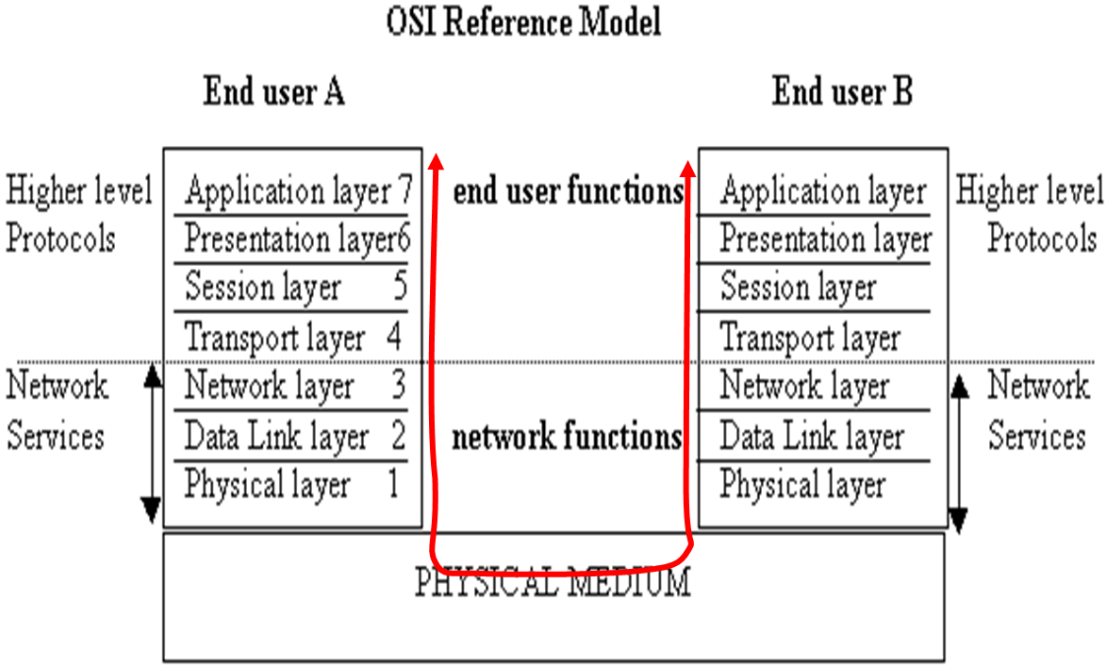
2) Resulting in the reduced complexity of the control system, project design engineering is made simpler, more efficient, and conversely less expensive.

3) By selecting a recognized and well-established system, this will make the Fieldbus equipment in your plant or plants interchangeable between suppliers.

4) The need to be concerned about connections, compatibility, and other potential problems is eradicated.

What constitutes a Fieldbus?

The specification of a Fieldbus should ideally cover all of the seven layers of the OSI model.



Fieldbus: OSI layer details

- Physical Layer [1] What types of signals are present, levels, representation of 1's and 0's, what type of media connects, etc.
- Link Layer [2] Techniques for establishing links between communicating parties.
- Network Layer [3] Method of selecting the node of interest, method of routing data.
- Transport Layer [4] Ensuring what was sent arrives at the receiver correcting any correctable problems.
- Session Layer [5] Not applicable to Fieldbuses.
- Presentation Layer [6] Not applicable to Fieldbuses.
- Application Layer [7] Meaning of data.
- The best way to cover layer 7 is to define standard profiles for standard devices.

What Fieldbus Networks are currently on the market?

Some of the Fieldbus technologies currently on the market

- AS-Interface (Europe)
- **CAN (German, Bosch, we will discuss in detail)**
- Interbus (German, Phoenix Contract)
- ModBus (America, Modicon)
- Profibus (German, Siemens)
- EtherNet (America, AB)
- Controlnet (America, AB)

Controller Area Network (CAN)

- Fast serial bus.
- Designed to provide an efficient and reliable link between sensors and actuators.
- CAN uses a twisted pair cable (dual-wire) to communicate at speeds up to 8Mbit/s (max) with up to 255 devices.
- CAN Bus supports multiple data rates – high data rate bus, low data rate bus.
- Advantages:
  - ✓ *Low cost*
  - ✓ *Handles high sensor data volumes with minimal latency.*
  - ✓ *Supports integration of advanced control systems, sensors, actuators, etc.*

Types of CAN

Properties	CAN 2.0	CAN FD	SAE J1939	ISOBUS (ISO 11783)	Automotive Ethernet
Year	1991	2011	1994	2001	2008
Standard	ISO 11898	ISO 11898	SAE J1708 / J1587	ISO 11783	IEEE 802.3 / BroadR-Reach
Devices	Up to 40	Up to 70	Up to 255	Up to 255	Thousands (based on topology)
Data Rate	Up to 1 Mbps	Up to 8 Mbps	Up to 1 Mbps	Up to 250 kbps	100 Mbps – 1 Gbps
DLC Rate	Fixed 8 bytes	Flexible 0–64 bytes	Flexible 0–64 bytes	Fixed 8 bytes	Up to 1500 bytes
Application	Light Vehicles	Light Vehicles	Heavy Vehicles	Heavy Vehicles	High-bandwidth control, Cameras

CAN Features

1) Any node can access the bus when the bus is quiet.

2) Non-destructive bit-wise arbitration to allow 100% use of the bandwidth without loss of data.

3) Variable message priority based on 11-bit / 29-bit packet identifier.

4) Peer-to-peer and multi-cast reception.

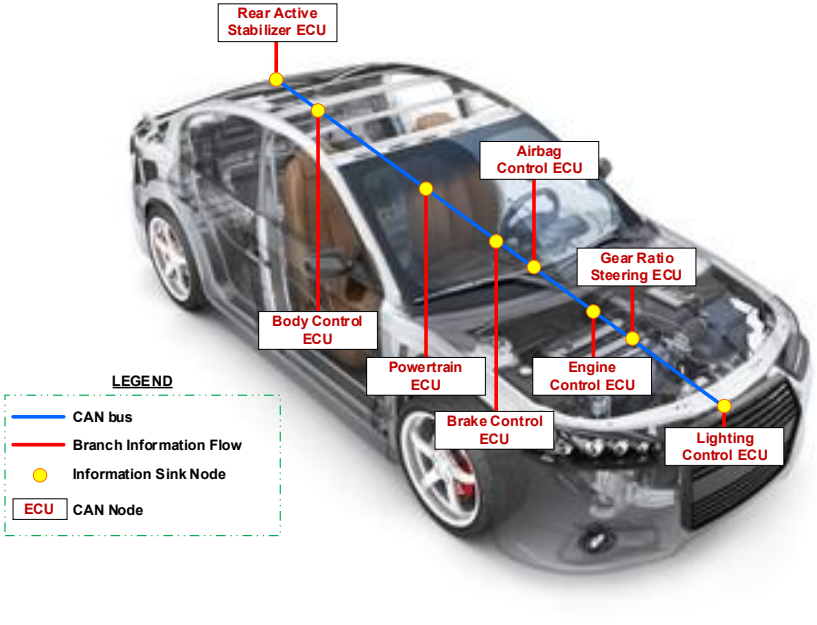
5) Automatic error detection, signaling and retries.

6) Data packets 4 or 8 bytes long.

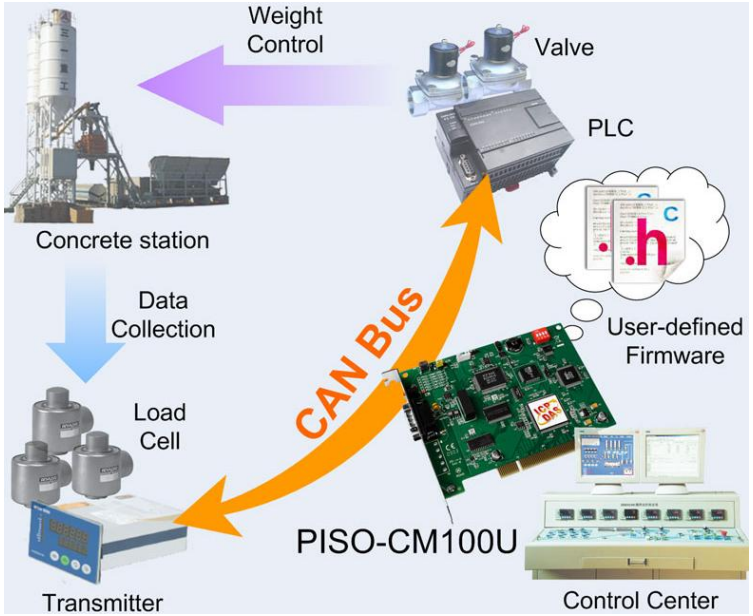
7) Asynchronous communication (Even Triggered).

Applications of CAN

Intra-vehicular communication

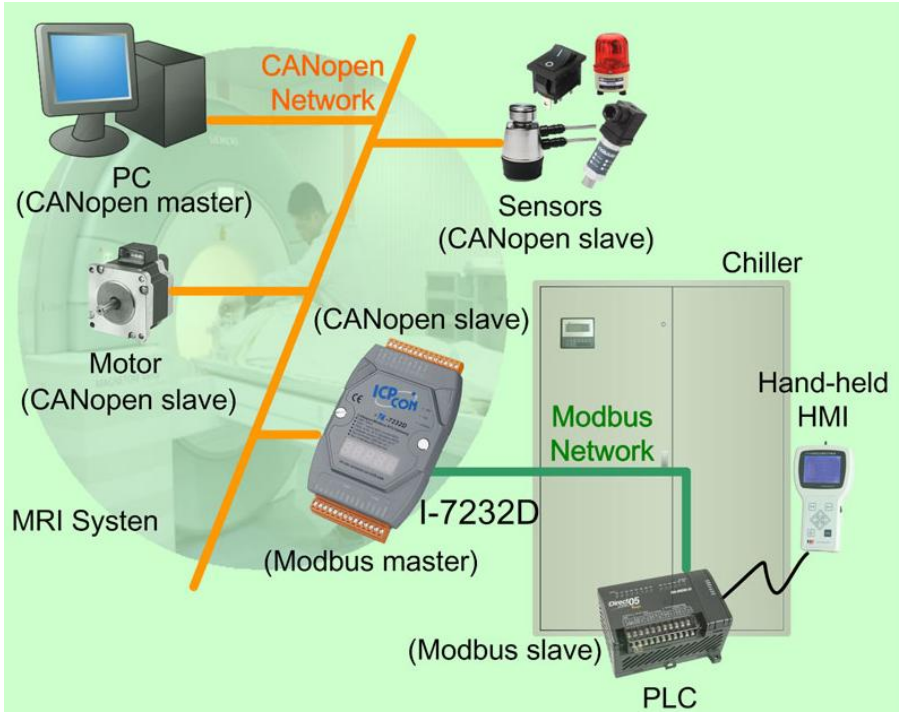


Concrete State Monitor & Control System

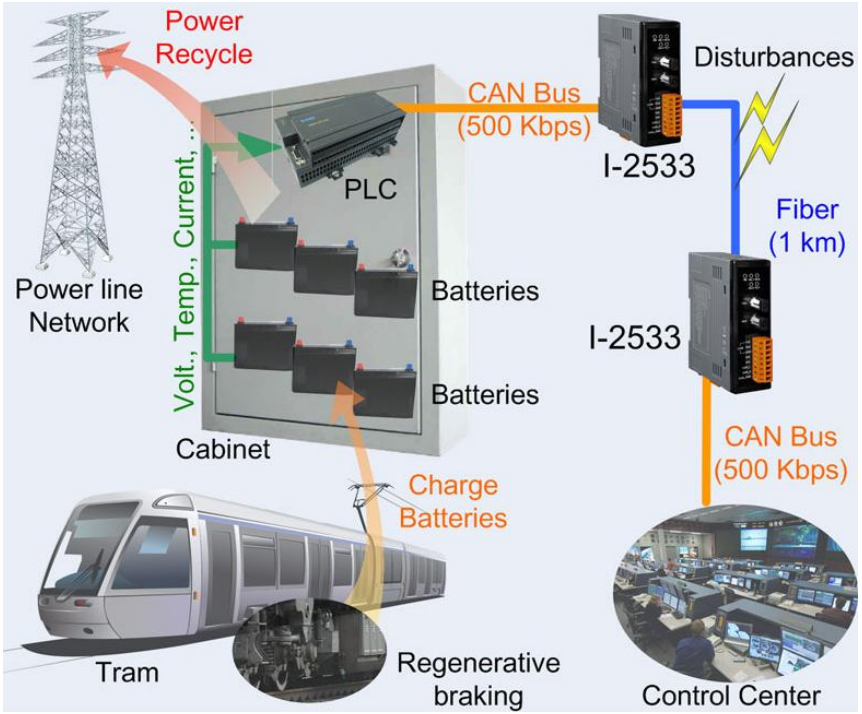


Applications of CAN

MRI Cooling System

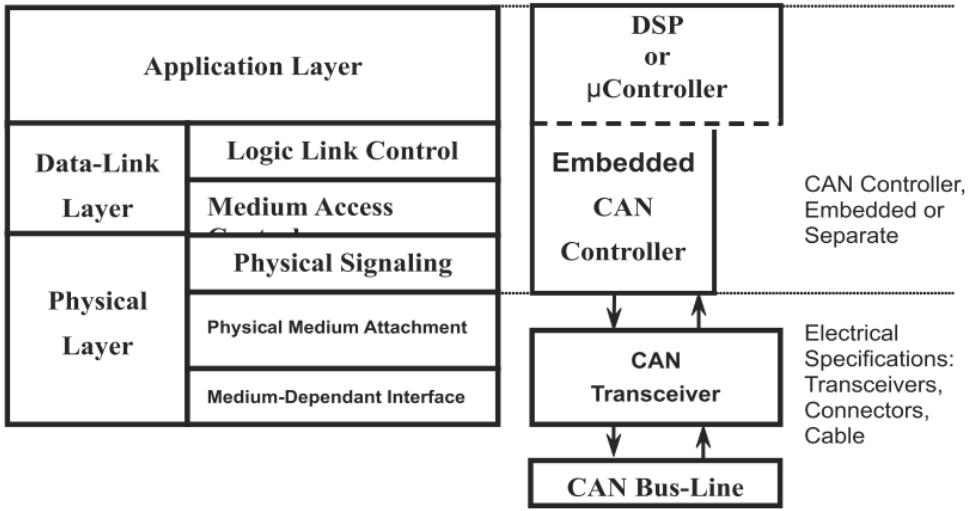


Tram Energy Recycle System

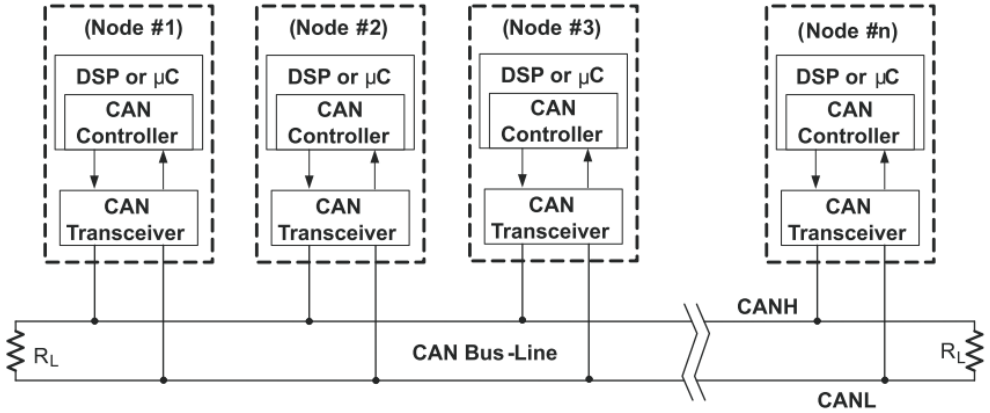


CAN Standard

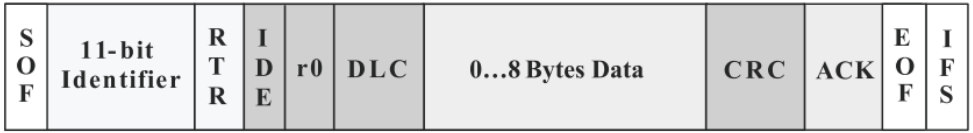
CAN Layered ISO 11898 Standard (conforming to OSI model)



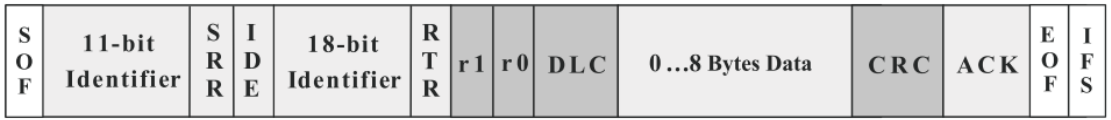
CAN Architecture



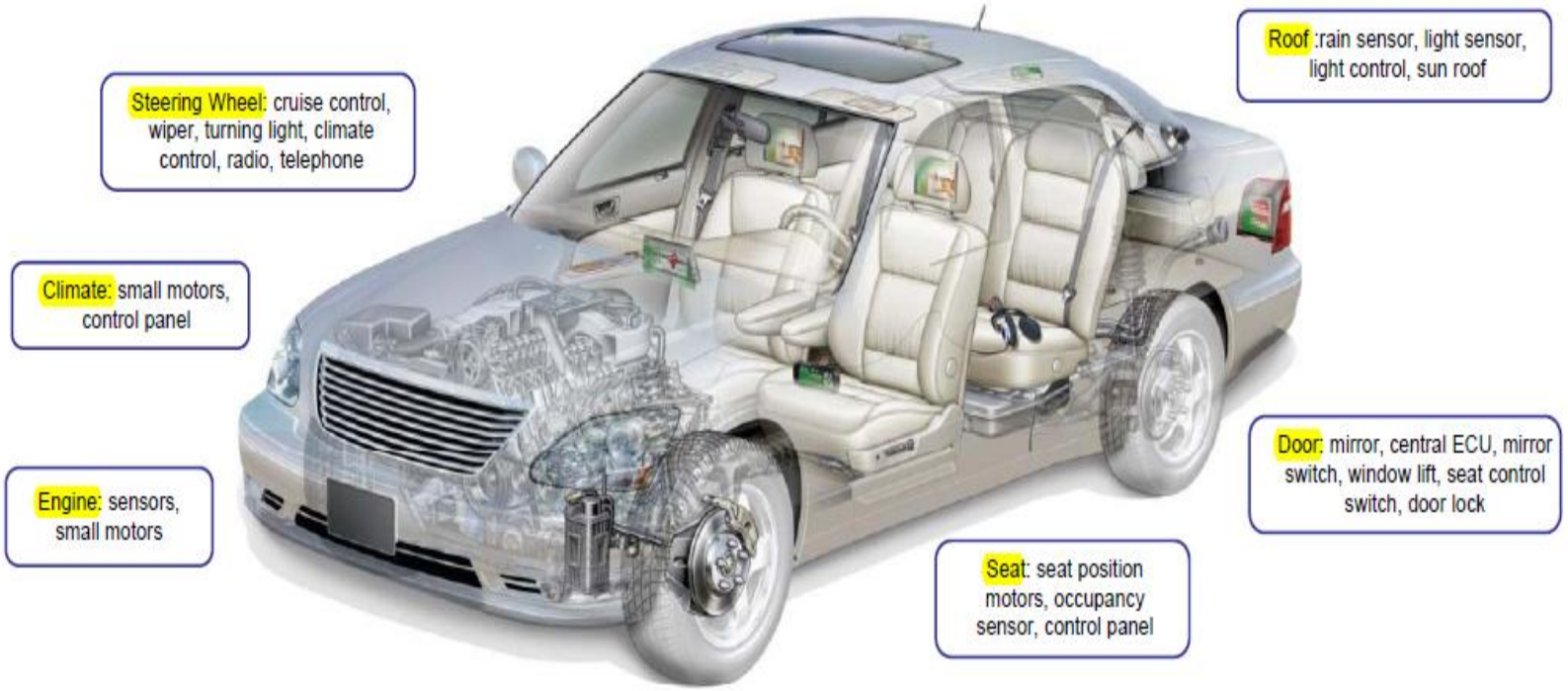
Standard CAN (11-bit)



Extended CAN (29-bit)

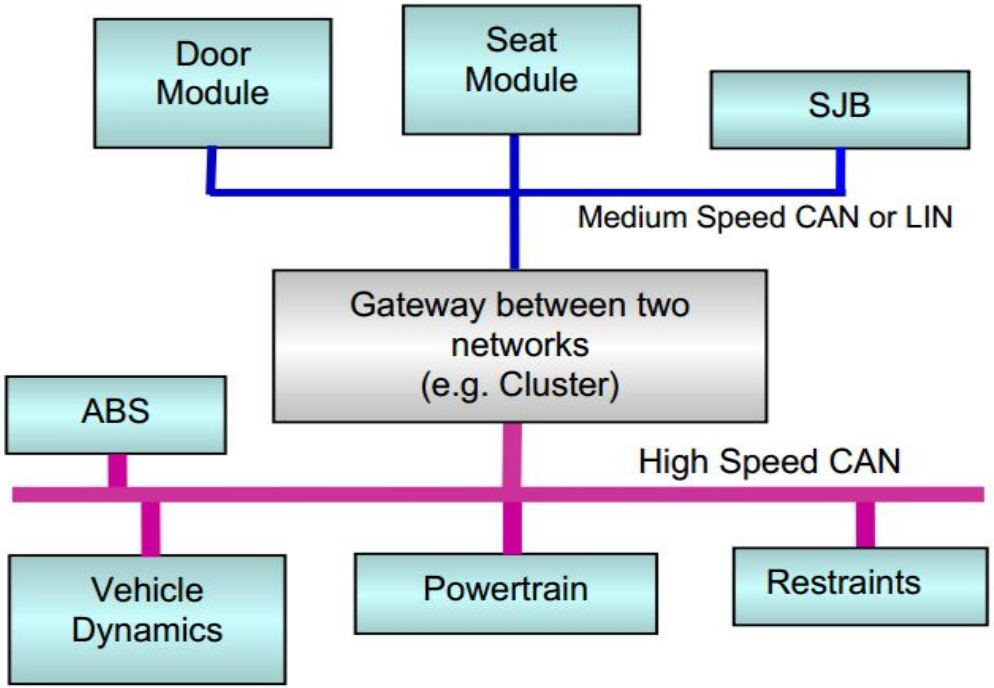


Working of the CAN Network: *Intra-vehicular Communication*

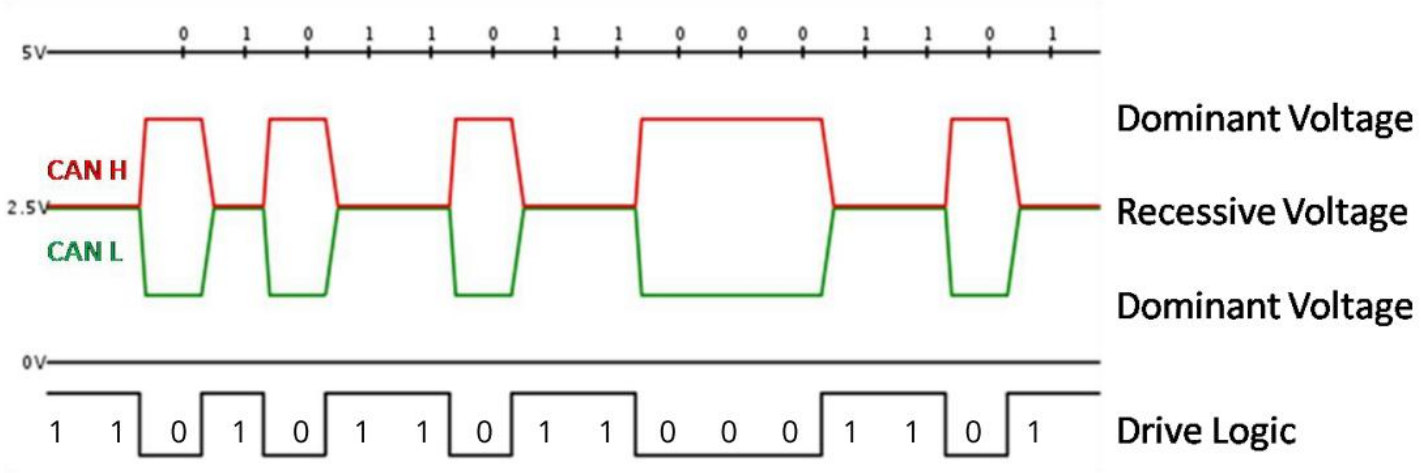


Working of the CAN Network: *Intra-vehicular Communication*

A schematic diagram of a current in-vehicle network



Working of the CAN Network: *CAN Message Structure*

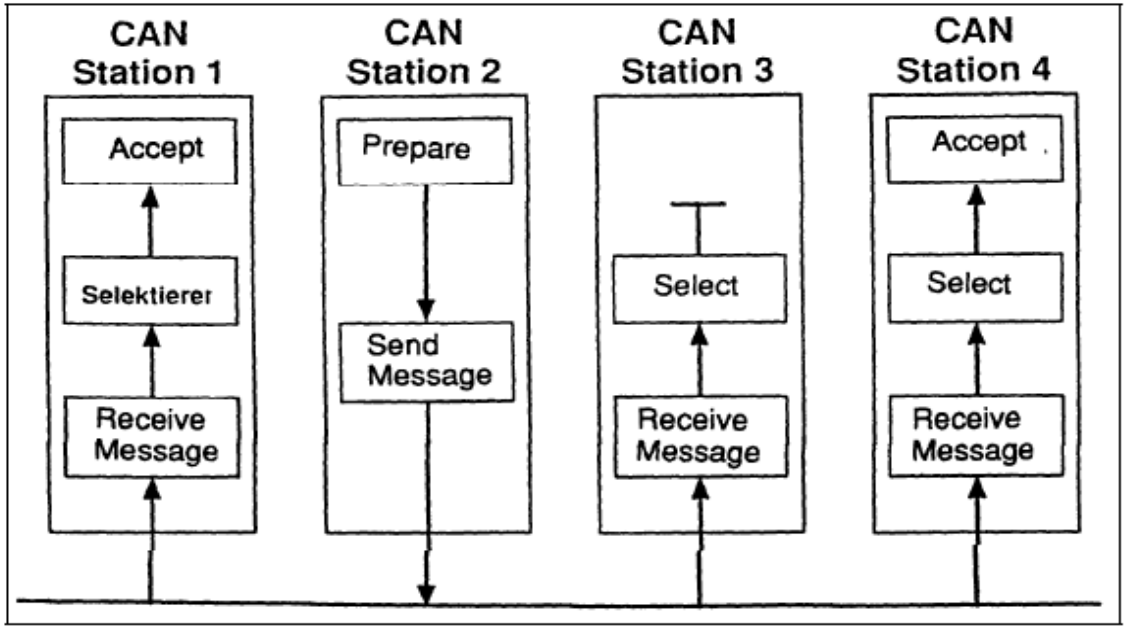


Two line communication allows us to reduce data loss due to noise.

Working of the CAN Network: *Transmission/Reception*

CAN bus versus point-to-point connections

- By introducing a single bus as the only means of communication, as opposed to the point-to-point network, we traded off channel access simplicity for circuit simplicity
- Since two devices might want to transmit simultaneously, we need to have a MAC protocol to handle the situation.
- CAN manages MAC issues by using a unique identifier for each of the outgoing messages
- The identifier of a message represents its priority.



Broadcast transmission and acceptance filtering by CAN nodes

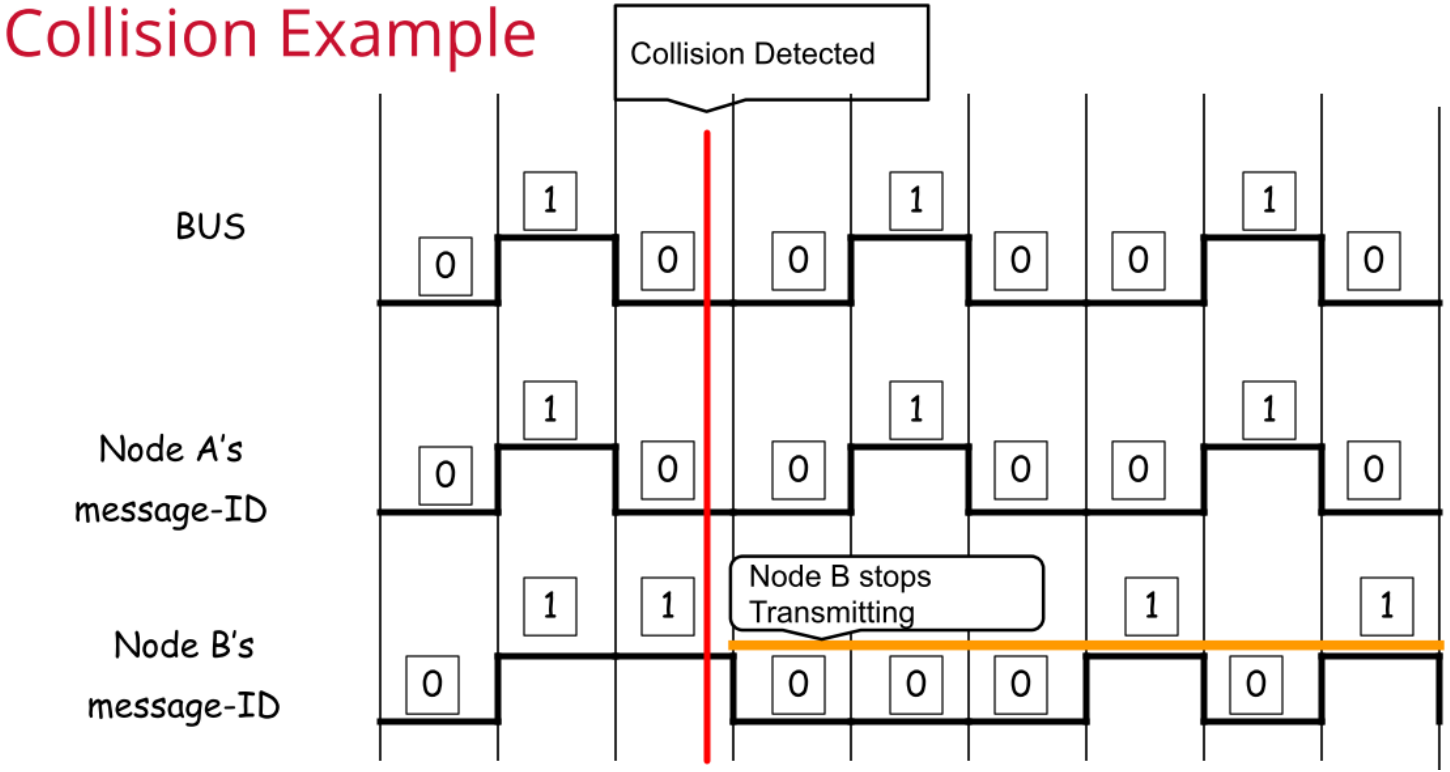
Working of the CAN Network: *Implicit collision handling*

- If two messages are simultaneously sent over the CAN bus, the bus takes the "logical AND" of all of them.

- Hence, the message identifiers with the lowest binary number get the highest priority.

- Every device listens on the channel and backs off when it notices a mismatch between the bus's bit and its identifier's bit.

Working of the CAN Network: *Implicit collision handling*



Project Demo

Title:
Implementation of real-time CANbus communication to study the protocol performance

Option:
Type 2: Implementation-oriented Project

Objectives:

- Development of a circuit using Arduino UNO and Raspberry Pi to implement a real-time CANbus communication architecture.
- The obtained CAN messages are then monitored to demonstrate the bitwise arbitration method for contention resolution.

Platforms & Tools

Hardware:

- *Raspberry Pi 3 Model B+*
- *Arduino UNO*
- *MCP2515 CANbus Transceiver Board*
- *10kΩ Potentiometer Sensor*

Software:

- *Raspbian OS (Linux & Python based) for RPi*
- *Arduino UNO IDE (Code written in C)*
- *SocketCAN Linux CANbus Driver Package*
- *Python Library – canutils, cantools, PyQt5*

Accessories:

- *Breadboard, Jumper cables, Peripherals*

Controller Area Network (CAN) Protocol

Features:

Peer-to-peer communication.
Short messages broadcasted over network.
Logic-high = 0, Logic-low = 1.
Robust noise immunity and fault tolerance.
Lacks data security and privacy

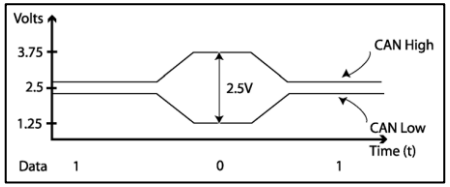


Figure-1: Differential voltage between CAN_H and CAN_L.

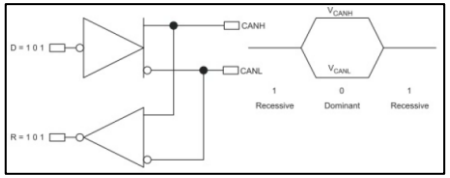


Figure-2: The inverted logic of a CAN bus.

CAN Bus I/O Characteristics	
CANbus Signal Type	Digital Interface
Output Voltage (High)	V_{OH} +4 volts min, +5.5 volts max
Output Voltage (Low)	V_{OL} +0 volts min, +1.5 volts max
Output Voltage	+16 volts (Absolute Max)
Output Current	100mA
Impedance	124 ohm termination between +/- terminals
Circuit Type	Differential
Bit Times	1uS @ 1Mb/s; 2uS @ .5Mb/s 4uS @ .25Mb/s
Encoding Format	Non-Return-to-Zero (NRZ)
Transmit/Receive Frequency	1Mb/s @ 40 meters
Topology	Point-to-Point
Medium	Shielded Twisted Pair (STP) @ 9 pin D-Sub
Access Control	Carrier Sense, Multiple Access with Collision Detect (CSMA/CD). Non-destructive bit wise arbitration

Figure-3: CAN Input/Output Characteristics.

Properties	CAN 2.0	CAN FD	SAE J1939
Year	1991	2011	1994
Standard	ISO 11898	ISO 11898	SAE J1708/J1587
Devices	Upto 40	Upto 70	Upto 256
Data Rate	Upto 1 Mbps	Upto 8 Mbps	Upto 1 Mbps
DLC Rate	Fixed 8 bytes	Flexible 0-64 bytes	Flexible 0-64 bytes
Application	Light Vehicles	Light Vehicles	Heavy Vehicles

Figure-4: Types of CAN protocols in Automotive Industry.

Bus Length (m)	Signaling Rate (Mbps)
40	1
100	0.5
200	0.25
500	0.10
1000	0.05

Figure-5: CAN 2.0 bus length v/s signaling rate.

Ref: https://www.ti.com/lit/an/sloa101b/sloa101b.pdf?ts=1670653821828&ref_url=https%253A%252F%252Fwww.google.com%252F

CANbus Bitwise Arbitration Method for Contention Resolution

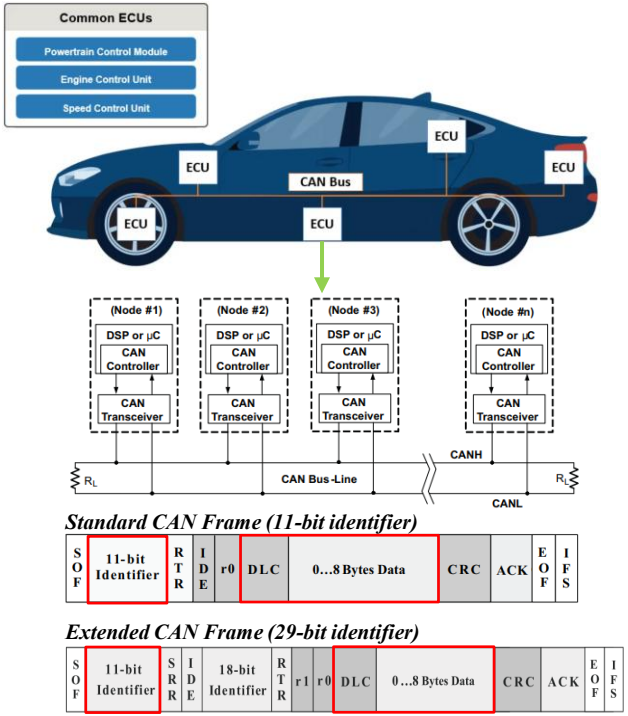


Figure-6: Illustration of CAN communication with two types of message formats.

- *A collision may occur when two or more nodes in the network are attempting to access the bus at the same time, which may result in delays or corruption of messages.*
- *CAN averts message/data collisions by using the message ID of the node, i.e. the message with the highest priority (= lowest message ID) will gain access to the bus, while all other nodes (with lower priority message IDs) switch to a listening mode.*

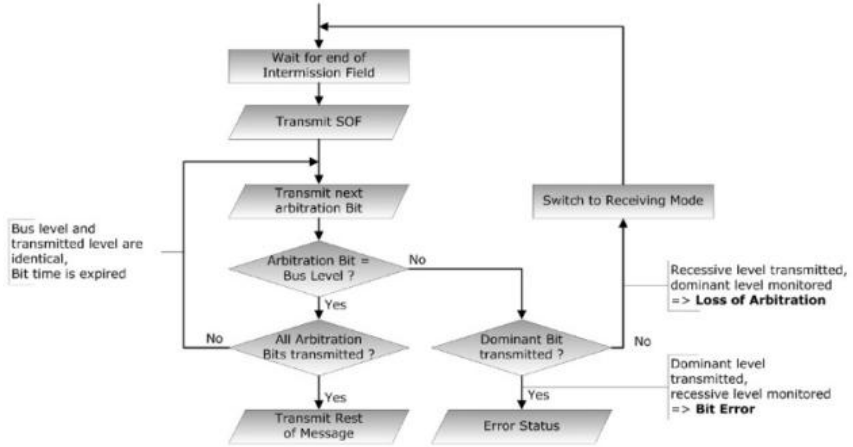
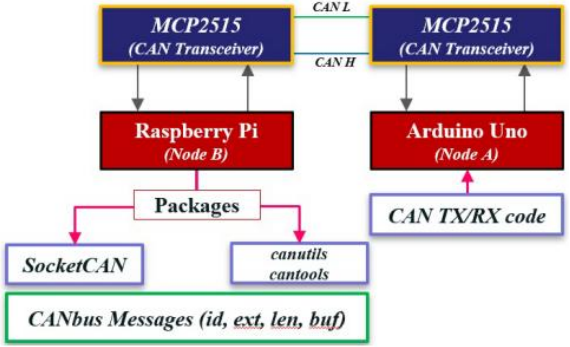


Figure-7: CAN bitwise arbitration flowchart.

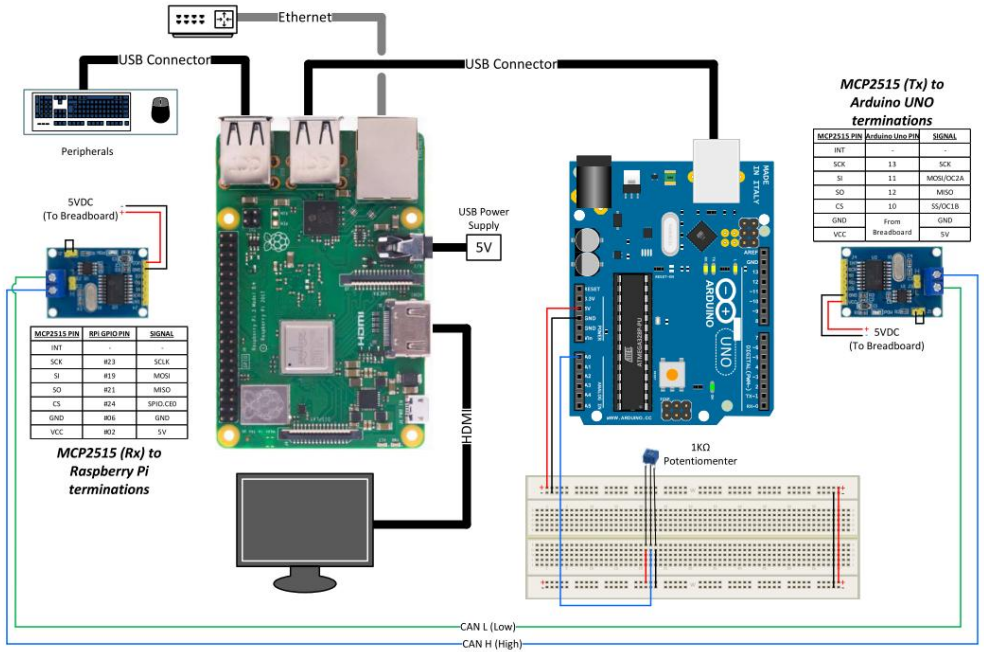
Ref:
(1) https://www.ti.com/lit/an/sloa101b/sloa101b.pdf?ts=1670653821828&ref_url=https%253A%252F%252Fwww.google.com%252F.
(2) https://copperhilltech.com/blog/controller-area-network-can-bus-bus-arbitration/

Conceptual Block Diagram



- *The bus network is built using a two-wire circuit comprising of CAN High and CAN Low.*
- *MCP2515 board is used to convert the CAN messages into SPI signals, and vice versa.*
- *The Arduino UNO and Raspberry Pi (acting as two nodes in the bus) are connected with the MCP2515 interface board (SPI-to-CAN).*
- *The Arduino UNO is programmed using C, and Raspberry Pi is programmed using Python.*
- *The real-time CAN messages are obtained on the Raspberry Pi, which acts as the HMI.*

Real-time CAN Communication Design Circuit



Experiment-I: Transmitting and Receiving CAN messages for each node

- *The objective was to ensure that the CAN communication has been configured correctly for each node within the bus.*
- *The Arduino UNO acts as Node A and Raspberry Pi acts as Node B.*
- *We were able to leverage the mcp2515 and SPI C-programming libraries to develop the code for Arduino UNO to enable transmission and reception of CAN messages.*

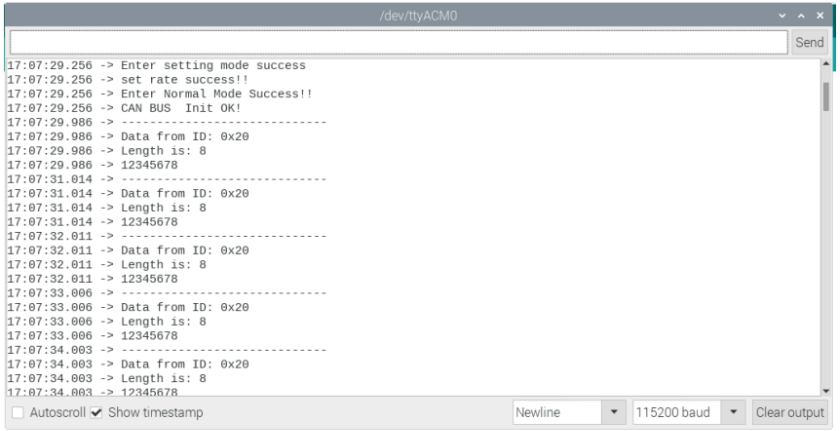


Figure-8: Experiment-I: Node B (TX) to Node A (RX). CAN frames are transmitted with constant data to test the connection.

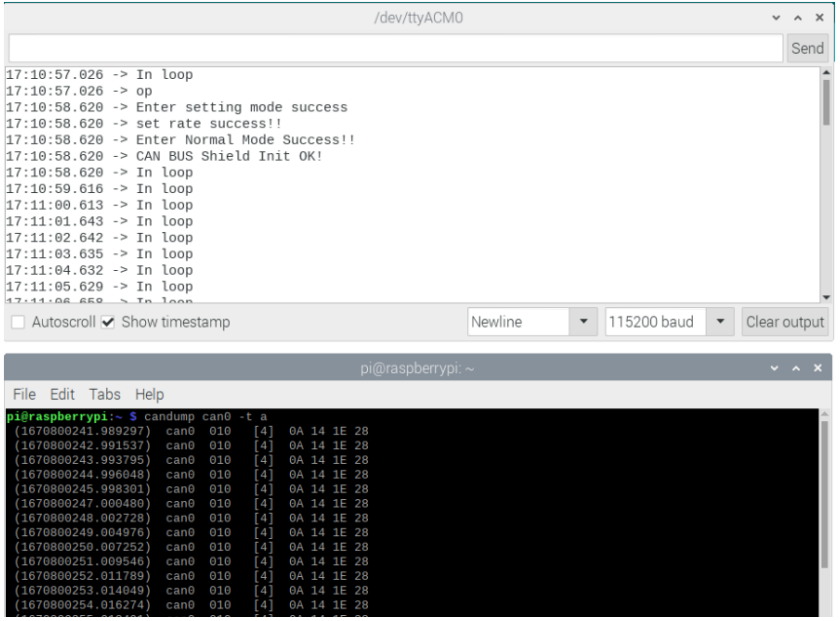


Figure-9: Experiment-I: Node A (TX) to Node B (RX). CAN frames are transmitted with constant data to test the connection.

Experiment-II: Replicating ECU transmission and reception over CANbus

- *For this experiment, we have designed Arduino UNO (Node A) to replicate an actual ECU of a vehicle.*
- *We have updated our initial circuit by adding a Potentiometer sensor.*
- *This sensor sends voltage signals to the Arduino which acts as an ECU.*
- *The sensor data is then converted into CAN frames and transmitted over the bus.*

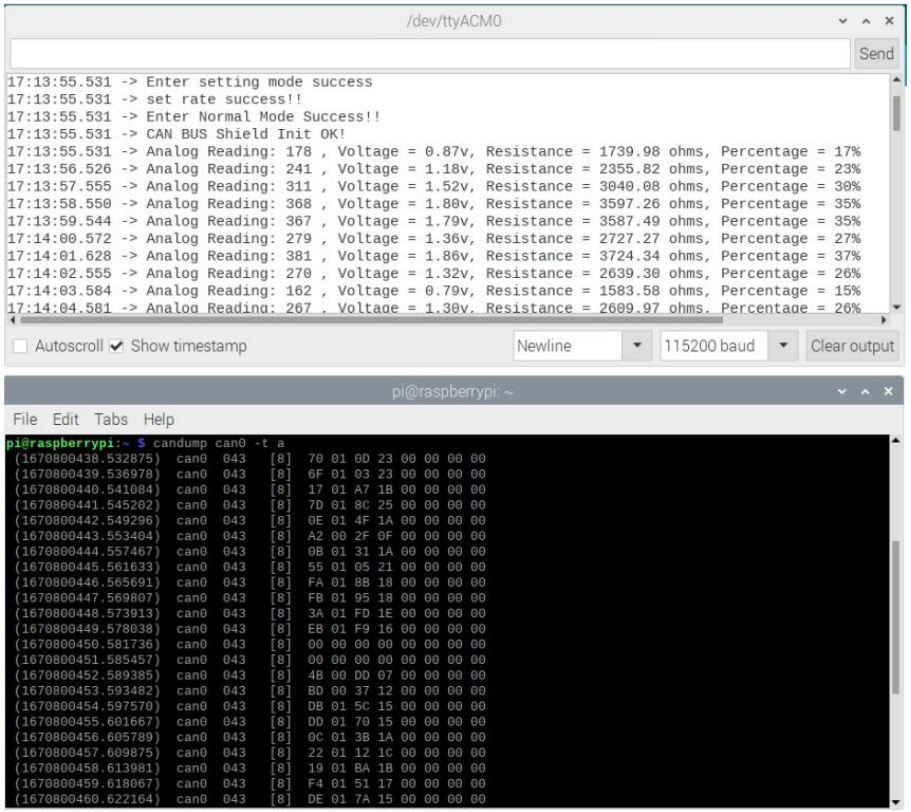


Figure-10: Experiment-II: Sensor data being transmitted over CANbus. As the the resistance of the Potentiometer changes, the voltage input to the Arduino UNO changes. Depending on the input signal, the CAN frames are created with the respective HEX values representing the data.

Experiment-III: Demonstrating the bitwise arbitration method for contention resolution

- *For this experiment, our objective was to demonstrate how the CAN protocol performs lossless bitwise arbitration method of contention resolution.*
- *The experimental setup from the earlier experiments have been updated to enable multiple nodes to transmit and receive CAN messages at the same time.*
- *We can observe that as we monitor the CAN traffic, the message with the lowest ID gets transmitted first, which is also indicated by the timestamp data. Therefore, we can understand that the bitwise arbitration method is successful in resolving collision between the messages.*
- *We had initially aimed to provide deeper insights into the arbitration method by visualizing the actual CAN frames and showing the bit-by-bit voltage graphs. While working on this project we realized that additional equipments, such as a CANlogger and an oscilloscope, are required to visualize the signals. The python libraries have limited capabilities for plotting CAN data.*

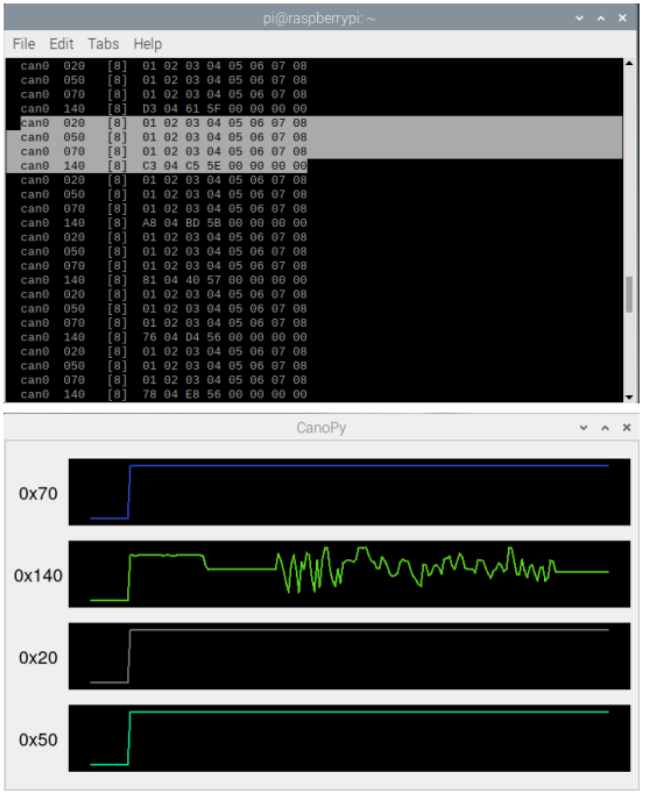


Figure-10: Experiment-III: CAN bitwise arbitration. Here, we can observe that the messages with the lowest ID gets transmitted first, as indicated by timestamp. Messages with ID 0x70, 0x20, and 0x50 are virtual CAN frames with constant data transmitted by RPi. Message ID 0x140 is the variable sensor data transmitted by Arduino UNO. Hence, the graph for 0x140 varies over time.

CAN Project Ideas

1) Two-node CAN (Arduino + MCP2515) with deadline monitoring
Objective – Measure end-to-end latency, bus utilization vs. deadline misses.
Resources – Arduino, Raspberry Pi, MCP2515, PICAN.

2) SocketCAN logger for jitter analysis
Objective – To time-stamp frames and compute jitter; To study scheduling & interrupt latency.
Resources – SocketCAN, can-utils, Raspberry Pi, MCP2515, PICAN.

3) FreeRTOS + CAN on STM32/ESP32
Objective – Implementing a multi-priority CAN node using FreeRTOS to analyze CAN message prioritization and RTOS task scheduling.
Resources – FreeRTOS, STM32/ESP32, SocketCAN, can-utils

Resources

Hardware:
- MCP2551 / MCP2561
- PICAN 2
- SparkFun CAN-BUS Shield
- Adafruit CAN Pal Transceiver

Software:
- SocketCAN
- can-utils
- STM32_CAN (for Arduino-STM32 core)
- Arduino CAN library
- Zephyr CAN APIs

Github:
- https://github.com/iDoka/awesome-canbus

https://bambooapps.eu/blog/can-bus-testing-alternative

References

[1] L. b. Othmane, L. Dhulipala, M. Abdelkhalek, N. Multari, and M. Govindarasu, “On the performance of detecting injection of fabricated messages into the can bus,” IEEE Transactions on Dependable and Secure Computing, vol. 19, no. 1, pp. 468–481, 2022.

[2] F. G. Tinetti, F. L. Romero, and A. D. P´erez, “Can bus experiments of real-time communications,” in Computer Science – CACIC 2017 (A. E.De Giusti, ed.), (Cham), pp. 253–262, Springer International Publishing, 2018.

[3] P. H. Nymann, “Can bus protocol: The ultimate guide (2022),” 3 2021.

[4] L. Dhulipala, “Detection of injection attacks on in-vehicle network using data analytics,” Master’s Thesis, Iowa State University, 2018.

[5] Antaira, “The basics of a fi eldbus network,” 4 2021.

[6] N. Velichkov, “How raspberry pi connects to can bus,” Oct 2021.

[7] S. Corrigan, “Introduction to the controller area network (can) application report introduction to the controller area network (can),” 2002.

[8] M. Falch, “Can bus explained - a simple intro (2022),” April 2022.

[9] W. Voss, “Controller area network (can bus) - bus arbitration,” November 2018.

[10] “Raspberry Pi 3 Model B+.” https://www.raspberrypi.com/.

[11] “Arduino UNO Rev3.” https://docs.arduino.cc/resources/.

References

[12] “MCP2515, Stand-Alone CAN Controller with SPI Interface.” https://ww1.microchip.com/downloads/en/DeviceDoc/MCP2515.

[13] “SocketCAN.” https://python-can.readthedocs.io/socketcan.

[14] “canutils.” https://github.com/linux-can/can-utils.

[15] “cantools.” https://github.com/cantools/cantools.

[16] “canopy.” https://github.com/Tbruno25/canopy.

[17] “PyQt5.” https://pypi.org/project/PyQt5/.

[18] “DBC Introduction.” https://docs.openvehicles.com/en/latest/.

Thank you

Questions?